

Assignment 4: Optimizing Transformer Translation with Ray Tune + Optuna

Name: Jyotin Goel

Roll Number: B22AI063

Objective

Optimize a custom PyTorch Transformer for **English -> Hindi** translation by replacing fixed hyperparameters with a **Ray Tune + Optuna** search workflow, and achieve baseline BLEU in fewer epochs.

Assignment Link

- Google Drive: <https://drive.google.com/drive/folders/19fhUruT03NkYp6u0uax-zjaZ6dZHUjzz?usp=sharing>
- Best Model (Hugging Face): https://huggingface.co/gjyotin305/assignment_4_model/tree/main

Submission Requirements

Push the following artifacts to your GitHub repository:

1. **B22AI063_ass_4_tuned_en_to_hi.ipynb** (or **.py**)
Refactored notebook/script with Ray Tune + Optuna implementation.
2. **B22AI063_ass_4_report.pdf**
1-2 page report including:
 - Baseline metrics (time, final loss, BLEU at 100 epochs)
 - 4+ tuned hyperparameters and their ranges
 - Best configuration found
 - Final metrics of best model (time, final loss, BLEU)
 - Epoch count required to match/beat baseline
3. **B22AI063_ass_4_best_model**
Best checkpoint from tuning sweep.

Grading Rubric

- Baseline Execution: **10%**
- Code Refactoring (Ray-compatible training loop): **20%**
- Hyperparameter Setup + Optuna Search: **30%**
- Efficiency Goal (match/beat baseline BLEU using $\leq X$ epochs): **20%**
- Report Quality: **20%**

Part 1: Baseline (Mandatory)

Run **en_to_hi.ipynb** without changing architecture/hyperparameters.

Record:

- Total training time for 100 epochs
- Final training loss
- Final BLEU score (from NLTK evaluation cell)

Keep baseline weights:

- `transformer_translation_final`

Baseline Log Template

Metric	Value
GPU	RTX A5000
Train Time (100 epochs, with <code>torch.compile</code>)	~30-40 minutes
Final BLEU (NLTK)	63.47 (<code>0.6347346879982754</code>)

Actual Run Results

Baseline (100 epochs, `torch.compile`)

- GPU: **RTX A5000**
- Time: **~30-40 minutes**
- BLEU (NLTK): **63.47**
- BLEU (raw): `0.6347346879982754`

After Optimization (Ray Tune + Optuna)

- BLEU (NLTK): **25.88**
- BLEU (raw): `0.2587952182172276`

Part 2: Refactor for Ray Tune + Optuna

2.1 Create Ray-compatible train function

Refactor to:

- `train_tune(config)`
- Initialize model/optimizer/criterion from `config`
- Report per-epoch metrics using:

```
ray.train.report({"loss": epoch_loss, "bleu": bleu_score, "epoch": epoch})
```

2.2 Define Hyperparameter Search Space (>= 4 params)

Suggested search space:

```
from ray import tune

param_space = {
    "lr": tune.loguniform(1e-5, 1e-3),
    "batch_size": tune.choice([16, 32, 64]),
    "num_heads": tune.choice([4, 8]),
    "d_ff": tune.choice([1024, 2048]),
    "dropout": tune.uniform(0.1, 0.4),
    "epochs": 40,
}
```

Note: Ensure `d_model % num_heads == 0`.

2.3 Configure Tuner with Optuna

```
from ray import tune
from ray.tune.search.optuna import OptunaSearch
from ray.tune.schedulers import ASHAScheduler

optuna_search = OptunaSearch(metric="loss", mode="min")
asha = ASHAScheduler(metric="loss", mode="min", max_t=40, grace_period=5)

tuner = tune.Tuner(
    train_tune,
    tune_config=tune.TuneConfig(
        search_alg=optuna_search,
        scheduler=asha,
        num_samples=20,
    ),
    param_space=param_space,
)

results = tuner.fit()
best_result = results.get_best_result(metric="loss", mode="min")
print(best_result.config)
```

Best Configuration Found

```
{
  "lr": 0.00012752212408379722,
  "batch_size": 16,
  "num_heads": 4,
  "d_ff": 1024,
  "dropout": 0.2810754004322514,
  "max_epochs": 40
}
```

Part 3: Efficiency Challenge

Goal:

- Match or exceed baseline BLEU (reference target ≥ 0.50) using **significantly fewer than 100 epochs** per trial.
- Cap each tuning trial at X epochs ($X < 100$).

Recommended:

- Use **ASHA** to early-stop weak trials
- Track both **loss** and **bleu**
- Save best checkpoint automatically

Expected Repository Structure

```
.
├── B22AI063_ass_4_tuned_en_to_hi.ipynb
├── B22AI063_ass_4_report.pdf
├── B22AI063_ass_4_best_model
├── en_to_hi.ipynb
├── tuned_train.py           # optional .py variant
└── README.md
```

Report Checklist (1-2 pages)

- Baseline metrics table
- Tuned hyperparameters + ranges
- Best configuration block
- Best model metrics vs baseline (time/loss/BLEU)
- Epoch efficiency comparison (**100** vs best trial epochs)
- Short conclusion: what mattered most in convergence

AI Assistance Disclosure

ChatGPT was used for:

- Documentation drafting/formatting
- Experiment reporting structure
- Boilerplate code skeletons for Ray Tune/Optuna setup

Model training, metric generation, and final conclusions were based on actual runs in this repository.